



POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

EXECUTIVE SUMMARY OF THE THESIS

Distributed Streaming Ensemble Methods in Renoir

LAUREA MAGISTRALE IN COMPUTER SCIENCE AND ENGINEERING - INGEGNERIA INFORMATICA

Authors: ANTONIO SULFARO (259954), ZHI QIANG SHI (252205)

Advisor: PROF. ALESSANDRO MARGARA

Co-advisors: PROF. GIANPAOLO SAVERIO CUGOLA, LUCA DE MARTINI, DARIO D'ABATE, FEDERICO GIANNINI

Academic year: 2025-2026

1. Introduction

High-velocity streaming workloads often exceed the throughput and memory capacity of single-machine online learners. To support streaming classification at scale, we develop a distributed streaming ensemble learning system by implementing three state-of-the-art algorithms—Adaptive Random Forest (ARF), Streaming Random Patches (SRP), and Aggregated Mondrian Forest (AMF)—on Renoir, a high-performance distributed dataflow runtime written in Rust. The design exploits the embarrassingly parallel structure of ensemble methods, distributing tree-level computation across worker nodes to bypass the single-threaded and JVM bottlenecks of existing frameworks. The system targets order-of-magnitude throughput gains across single and multi-core networked deployments, while strictly preserving the mathematical integrity and predictive accuracy of the centralized baselines.

Keywords: Streaming Machine Learning (SML), ensemble methods, distributed dataflow, ARF, SRP, AMF, Renoir, Rust, concept drift.

2. Background

2.1. Streaming Machine Learning

Streaming Machine Learning (SML) operates on continuously arriving and potentially unbounded data streams under strict computational constraints. Unlike traditional batch learning, where data can be stored and processed repeatedly, SML learns incrementally from incoming instances while maintaining bounded memory usage and processing time. Models must therefore satisfy single-pass processing and anytime prediction requirements, often relying on compact sufficient statistics rather than retaining historical observations. A further challenge is **concept drift**, where the underlying data distribution changes over time and may degrade predictive performance if not addressed. To mitigate this, various drift detection methods are employed, allowing the single learner to continuously update its parameters or structure to accommodate the evolving patterns. In this work, model performance is evaluated using *prequential* (interleaved test-then-train) *evaluation*, which provides a continuous estimate of predictive accuracy throughout the stream. Raw data streams are additionally preprocessed through missing-value imputation, categorical encoding, and min-max normalization before learning.

Ensemble Methods. Ensemble methods leverage multiple diverse base learners, such as random trees, to enhance model performance. To prevent redundant correlation, diversity is stochastically induced via several methods, including bagging, feature subsampling, and mon-drian process. Individual predictions are resolved through three strategies: Majority Voting calculates the mode of discrete class predictions with equal weight; Weighted Voting scales these discrete votes by performance-based weights; and Soft Voting averages predicted class probabilities across all learners to produce the final prediction.

2.2. Hoeffding Tree

Hoeffding Trees are the foundational algorithmic paradigm for streaming decision trees, incrementally inducing a model by leveraging the Hoeffding bound to guarantee statistical convergence to a traditional batch learner. Rather than caching raw historical observations, active leaves maintain compact sufficient statistics to evaluate the structural merit $G(\cdot)$ of potential attribute splits. To process continuous features under strict memory constraints, leaves employ either Gaussian or histogram-based density estimators.

To minimize the computational bottleneck of continuous split evaluations, a grace period parameter n_{min} restricts heuristic evaluations until a minimum number of instances accumulate at the leaf. Once reached, a split is executed on the highest-performing attribute X_a over the second-best candidate X_b if the difference in their information gain exceeds the calculated Hoeffding bound, expressed as: $\Delta G > \epsilon = \sqrt{\frac{R^2 \ln(1/\delta)}{2n}}$ where R is the range of the target variable, $1 - \delta$ is the user-defined confidence, and n is the accumulated instance count.

Hoeffding Tree Based Forest Hoeffding Tree Based Forests, such as Adaptive Random Forest (ARF) [2] and Streaming Random Patches (SRP) [3], employ ensembles of Hoeffding Trees to improve predictive performance on evolving data streams. Both methods induce diversity through online bagging, using Poisson-distributed instance weights ($k \sim \text{Poisson}(\lambda)$), and random feature subspaces. ARF applies feature randomization locally at each node during split evaluation, whereas SRP assigns each learner a fixed feature subspace at initialization. To adapt to concept drift, both methods rely on ADWIN-

based warning and drift detectors: a warning signal initiates a background learner, while a confirmed drift replaces the active model.

2.3. Mondrian Tree

Mondrian Trees employ a non-parametric geometric approach to partition data streams, representing nodes as hyperrectangles bounded by the coordinate-wise extremes of observed features. Space is recursively divided through the Restricted Mondrian Process, which confines axis-aligned guillotine splits to the empirical data range and avoids empty leaves. Unlike statistical decision trees, splits are stochastic events independent of class labels; when an incoming sample extends a node’s boundaries, the split intensity is proportional to the corresponding spatial extension. Class probabilities are estimated using Dirichlet-based smoothing to mitigate zero-frequency issues. During inference, Context Tree Weighting hierarchically combines local node predictions with subtree predictions through dynamically updated weights. These weights are optimized online using logarithmic loss, allowing the model to continuously adapt as the data distribution evolves.

Mondrian Tree Based Forest Aggregated Mondrian Forests (AMF) [4] combine class probability distributions produced by individual Mondrian Trees through Soft Voting.

2.4. The Renoir Dataflow Runtime

Renoir [1] is a Rust-based distributed stream processing framework that represents computations as directed dataflow graphs, where operators transform data streams flowing from sources to sinks. The platform combines a high-level programming model with low runtime overhead through Rust’s compile-time optimizations and memory-safe execution. Computations are expressed through stateless operators (`map`, `flat_map`, `filter_map`, `drop_key`) and stateful operators (`rich_map`, `rich_map_transient`). To support scalable execution, Renoir exploits both task and data parallelism, using routing primitives such as `group_by` to partition data deterministically by key and `repartition_by` to alter forwarding strategies or constrain execution to specific hosts.

3. Design and Implementation

The proposed system combines the world of streaming ensemble methods with the scalability mechanisms provided by the Renoir engine.

3.1. Software Architecture

The system is built as a generalized, extensible framework for online ensemble learning implemented in Rust to maximize streaming throughput and ensure native compatibility with the Renoir dataflow runtime.

The architecture, as illustrated in Figure 1, balances abstraction, efficiency, and modularity through a decoupled, multi-layer design: an **abstraction layer** defining a standardized `OnlineLearnerTrait` interface, an intermediate **generalization layer** (`HTBasedLearner`) capturing shared behaviors of Hoeffding-tree families, and an **implementation layer** where specific algorithmic logic is injected. This decoupled strategy facilitates a "plug-and-play" paradigm, allowing other possible base learners to hook directly into the system without altering the core dataflow logic.

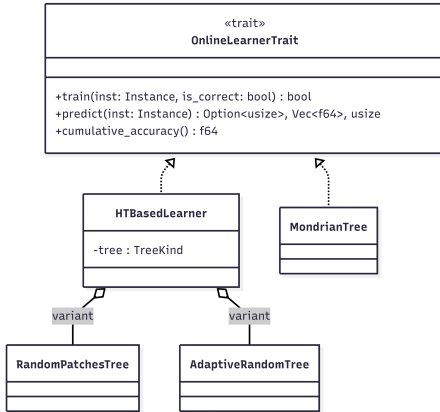


Figure 1: High-level class diagram of the learner abstraction.

To minimize cross-node synchronization overhead, the architecture eliminates centralized master nodes by managing data streams via stateful, per-partition processing and key-based routing. Within parallel processing nodes, high-performance polymorphism is achieved via Rust’s Algebraic Data Types (enums). This design pattern ensures compile-time sizing, enforces contiguous memory allocation that maximizes CPU L1/L2 cache locality, and guarantees a seamless serialization boundary for shifting model states

across parallel Renoir task partitions.

HTBasedLearner State management within the intermediate `HTBasedLearner` layer encapsulates online bagging via localized pseudo-random number generation, alongside a drift-driven state machine managed by a dual-state `ADWIN` detector. Concept drift triggers the non-blocking allocation of background companion trees that warm up sufficient statistics in parallel using localized feature subspace strategies, executing in-place structural promotion or false-alarm pruning to guarantee bounding memory footprints.

3.2. The Streaming Pipeline

The system leverages Renoir to exploit horizontal parallelism by treating each tree in the ensemble as an independent learner. To bypass centralized master bottlenecks, the architecture decouples instance distribution from prediction aggregation through a stream re-keying strategy that orchestrates parallelization natively within the dataflow model.



Figure 2: Logical stages of the ensemble pipeline.

As illustrated conceptually in Figure 2, the processing pipeline is structured into four logical stages and one offline test harness in Python designed to automatically evaluate performance and draw useful plots.

Stage 1: Input and Initialization. A single entry source node ingests the streaming dataset, assigns a monotonic `instance_id` to each observation, and executes a `flat_map` operation to replicate the instance into N copies, generating a stream of tuples keyed by a unique `tree_id`.

Stage 2: Per-Tree Processing. The stream is partitioned via `group_by(tree_id)` and routed to stateful `rich_map` operators where individual models evaluate and train on the instances independently in a test-then-train manner.

Stage 3: Per-Instance Aggregation. Here we shift the coordinate space by dropping the tree key and applying a `group_by(instance_id)`. A `rich_map_transient` operator accumulates incoming tree votes; once all N tree responses for a given identifier are collected, the final ensemble prediction is resolved, and the local state is

instantly purged to maximize memory efficiency. This asynchronous collection optimizes system throughput by avoiding serialization order stalls.

Stage 4: Output. We drop the instance key and invoke `repartition_by(Replication::One)` to route the un-ordered prediction results to a single node, which formats and records the stream using an optimized CSV sink operator.

4. Experiments and Results

This section synthesizes the empirical evaluation of the system, directly addressing the core research questions of our work.

RQ1: Comparison to the baseline The Rust-based implementations of ARF, SRP, and AMF within the Renoir framework achieve strict predictive *accuracy parity* with their centralized reference baselines (MOA and onelearn), with deviations well within expected bounds from floating-point arithmetic, PRNG divergence, and non-deterministic distributed instance ordering. In sequential settings, as highlighted in Table 1, single-core profiles reveal *massive performance gains*, yielding speedup factors from $1.97\times$ to $35.73\times$ over MOA and up to $18.24\times$ over onelearn. These gains are driven by Rust’s zero-cost abstractions, bounds-check elision, and cache-localized memory management, bypassing heavy JVM heap allocations and garbage collection.

Scaling the workload Moving to a parallel multi-core configuration (100 trees across 32 cores), which is only supported by ARF among the baselines, the evaluation exposes a severe *concurrency ceiling* in the baseline framework. As we can see in Table 2, under high thread counts, MOA’s performance degrades drastically due to shared-memory lock contention and aggressive JVM garbage collection routines across 32 threads, culminating in fatal out-of-memory crashes or timeouts on high-dimensional streams like **Spam**. Conversely, Renoir’s decoupled, shared-nothing dataflow model scales seamlessly without centralized bottlenecks, amplifying its absolute speedups factor from $6.01\times$ to a maximum of $21.82\times$ on the **AGR(a)** benchmark while maintaining absolute memory stability.

Remote cluster deployments In a remote environment across multiple nodes, the system

Dataset	Speedup (ARF)	Speedup (SRP)	Speedup (AMF)
Airlines	35.73	30.01	1.63
Elec	9.89	13.29	18.24
Covtype	7.83	12.99	4.88
Kdd99	3.07	3.72	8.64
ADS	9.84	24.39	1.04
Nomao	7.78	12.88	3.12
Spam	29.29	31.00	0.87
LED(a)	1.97	4.17	7.19
LED(g)	2.01	4.05	7.20
AGR(a)	11.04	14.35	5.80
AGR(g)	10.97	15.91	5.80
RBF(m)	4.40	7.85	5.36
RBF(f)	4.35	8.26	5.33

Table 1: Speedup comparison - using 10 base models and 1 core - with the respective baseline.

Dataset	Throughput (inst/s)	Speedup
Airlines	171,367	20.96
Elec	159,786	13.53
Covtype	64,560	12.39
Kdd99	79,746	14.36
ADS	12,942	10.65
Nomao	45,880	9.23
Spam	857	∞
LED(a)	36,385	6.01
LED(g)	38,080	6.14
AGR(a)	188,841	21.82
AGR(g)	169,939	20.55
RBF(m)	81,135	11.16
RBF(f)	79,911	9.89

Table 2: ARF raw throughput and speedup comparison with MOA - 100 base models and 32 cores

reveals an explicit trade-off between computational density and communication overhead that is heavily governed by data dimensionality. On low-dimensional streams (e.g., **Airlines**, **LED**), Renoir preserves over 90% of its local parallel execution efficiency because tree-level processing heavily dominates minimal network costs. In hyper-dimensional environments (**ADS**, **Spam**), cross-node efficiency drops to 35.03% and 26.99% as large-vector serialization and network coordination emerge as the primary bottlenecks. Overall, despite this communication penalty, Renoir’s absolute distributed throughput consistently outperforms MOA’s baseline by orders of magnitude, validating the architecture’s viability for distributed streaming clusters.

RQ2: Multi-Core Scalability The scalability analysis tracking execution scaling from 1 to 32 cores confirms that the decentralized aggre-

Dataset	Aggregated Mondrian Forest (20 trees)					Hoeffding Tree Based (100 trees)			
	2 core	4 core	8 core	16 core	32 core	ARF 16 core	ARF 32 core	SRP 16 core	SRP 32 core
Airlines	1.92	3.45	5.10	8.33	11.59	7.63	12.06	7.81	12.35
Elec	1.94	3.67	5.89	8.26	11.04	7.44	11.84	7.33	11.01
Covtype	2.00	4.02	6.55	9.60	15.81	7.82	11.81	8.00	11.52
KDD99	1.77	3.42	5.51	8.05	9.87	7.78	8.61	8.27	11.74
ADS	1.96	3.23	3.91	5.26	5.95	7.22	9.15	7.57	9.66
Nomao	1.94	3.93	6.50	8.90	13.20	7.47	11.17	8.13	11.65
Spam	1.80	2.56	2.75	2.88	2.92	5.62	5.41	5.95	5.57
LED(a)	1.92	3.52	5.42	7.37	10.39	8.02	12.10	7.79	11.39
LED(g)	1.92	3.50	5.48	7.52	10.49	7.82	11.84	7.68	11.42
AGR(a)	2.00	3.86	6.12	9.02	14.98	7.79	12.16	7.97	12.51
AGR(g)	2.02	3.88	6.07	9.02	14.98	7.90	12.03	8.10	12.99
RBF(m)	2.01	3.96	6.53	10.25	17.66	8.29	12.90	8.18	12.70
RBF(f)	2.02	3.96	6.47	10.06	16.78	7.92	12.42	7.94	12.22

Table 3: Scalability Speedup Ratios for Mondrian and Hoeffding tree based ensembles.

gation pipeline effectively mitigates the many-to-one bottlenecks common to traditional centralized voting stages. On physical cores (up to 16 threads), the system maintains a parallel efficiency near 50%. When scaling to 32 logical threads via Simultaneous Multithreading, the system yields an absolute throughput gain of an order of magnitude over centralized baselines, outperforming parallel MOA’s efficiency on the RBF(m) benchmark by a factor of three (40.3% vs 12.22%).

However, the empirical evaluation exposes clear algorithmic and hardware boundaries:

- **Algorithmic Limits:** The Mondrian-based AMF exhibits a scalability plateau at lower thread counts due to its tight 20-tree allocation limit, which inherently leaves available execution units idle.
- **Hardware Limits:** Ultra-high-dimensional datasets (e.g., **Spam** with 39,917 features) experience a throughput efficiency regression when crossing the physical-to-logical core boundary.

The main results can be seen in Table 3.

RQ3 & RQ4: Structural Trade-Offs against Distributed VHT A direct architectural comparison against the state-of-the-art distributed Vertical Hoeffding Tree (VHT) exposes a fundamental operational trade-off. ARF and SRP consistently dominate VHT in predictive accuracy under non-stationary distributions, correcting for catastrophic accuracy drops (e.g., VHT dropping to 58.81% on **Covtype** due to its lack of explicit drift detection). While VHT exhibits superior raw throughput on stationary, low-dimensional streams, due to its leaner single-tree architec-

ture, this advantage rapidly degrades as feature dimensionality scales up, encountering severe vertical partitioning and coordinated split-decision coordination bottlenecks.

Crucially, tracking the framework transition from MOA to Renoir validates a major architectural insight: while migrating VHT yields a modest $1.99\times$ to $3.14\times$ performance increase, migrating ARF triggers a massive $6.01\times$ to $21.82\times$ throughput amplification. This proves that ensemble methods are inherently better aligned with the distributed dataflow paradigm than single-tree architectures. The embarrassingly parallel independence of base learners allows Renoir to maximize concurrent pipelines without synchronization penalties, establishing this framework-level contribution as a highly impactful advancement in distributed stream mining.

RQ5: Hyperparameter Analysis and Optimization To provide a deeper insight into the practical utility of these models, this research extends beyond baseline comparisons to systematically evaluate how each algorithm behaves under varied operational configurations. Specifically, the ensemble configurations detailed in Table 4 demonstrate that tree count acts as the primary determinant of computational throughput. Transitioning from 10 to 100 trees incurs an average penalty of 315,892 instances per second while providing a marginal predictive accuracy gain of only +0.24%. For AMF, scaling beyond 15 trees accelerates throughput degradation from 1% to 2.5% per tree, facing strict diminishing returns as accuracy almost completely plateaus past 50 trees.

Conversely, optimizing the feature patch size

Hoeffding Tree Based						Mondrian Tree Based			
Dataset	#Tree	Patch	λ	Accuracy	Throughput (inst/s)	Dataset	#Tree	Accuracy	Throughput (inst/s)
ADS	10	60.0	1	98.66%	40,508	covtypeNorm	10	92.49%	119,796
	10	60.0	6	99.81%	39,965		15	92.77%	111,948
	10	2.6	1	98.69%	42,930		20	92.88%	90,219
	10	2.6	6	99.33%	41,423		30	93.09%	63,153
	100	60.0	1	98.81%	17,170		40	93.15%	39,205
	100	60.0	6	99.82%	12,913		50	93.21%	34,687
	100	2.6	1	98.66%	35,542		60	93.24%	29,811
	100	2.6	6	99.33%	34,696		70	93.29%	22,652
LED(g)	10	62.5	1	73.02%	701,583	RBF(f)	10	44.71%	124,533
	10	62.5	6	73.10%	280,667		15	46.69%	112,360
	10	20.8	1	72.17%	1,084,280		20	47.96%	94,787
	10	20.8	6	72.51%	777,702		30	49.49%	69,109
	100	62.5	1	73.13%	89,927		40	50.42%	39,730
	100	62.5	6	73.17%	38,390		50	51.05%	35,249
	100	20.8	1	73.15%	176,247		60	51.51%	29,762
	100	20.8	6	73.16%	77,041		70	51.84%	21,791

Table 4: Performance metrics across different datasets and hyperparameter configurations.

($m = \sqrt{\text{features}}$) drastically minimizes split-evaluation costs for Hoeffding-based ensembles, preserving global predictive accuracy while recovering massive throughput. The Poisson re-sampling parameter (λ) offers the most optimal cost-to-benefit profile, generating the largest isolated accuracy improvement (+0.49%) with minimal throughput penalties. Consequently, for velocity-critical applications, the optimal operational strategy demands minimizing ensemble sizes and feature patches while inflating λ to structurally preserve learner diversity.

5. Conclusion

This thesis makes two distinct contributions. The **engineering contribution** is a production-quality, fully validated distributed streaming ensemble learning system in Rust on Renoir, covering ARF, SRP, and AMF, as three leading SML ensemble algorithms, in a single unified, extensible framework.

The **scientific contribution** is the empirical demonstration, across 13 datasets and five experimental dimensions, that ensemble methods are structurally better suited to the distributed dataflow model than single-tree approaches: the embarrassingly parallel independence of base learners maps directly onto Renoir’s shared-nothing pipeline, amplifying the framework-level speedup by an order of magnitude beyond what prior distributed decision-tree work achieved.

The system is ready for high-velocity production workloads: it preserves predictive accuracy, scales to arbitrary core counts, survives

extreme dimensionality where Java-based frameworks crash, and outperforms the state of the art at every tested scale.

Crucially, while expanding ensemble size may yield diminishing returns, this high-throughput architecture natively enables an effortless shift toward multi-tenant orchestration, allowing idle cores to be easily reallocated for critical background tasks, like real-time drift tracking or background learner training, without rewriting the pipeline.

References

- [1] Luca De Martini, Alessandro Margara, Gianpaolo Cugola, Marco Donadoni, and Edoardo Morassutto. The renoir dataflow platform: Efficient data processing without complexity. *Future generation computer systems*, 160:472–488, 2024.
- [2] Heitor Murilo Gomes, Jesse Read, and Albert Bifet. Adaptive random forests for evolving data stream classification. pages 240–249, 10 2017.
- [3] Heitor Murilo Gomes, Jesse Read, and Albert Bifet. Streaming random patches for evolving data stream classification. pages 240–249, 11 2019.
- [4] Jaouad Mourtada, Stéphane Gaïffas, and Erwan Scornet. Amf: Aggregated mondrian forests for online learning. *Journal of the Royal Statistical Society Series B: Statistical Methodology*, 83(3):505–533, 2021.